WEEK 1

Week 1: Local Infrastructure with Vagrant - Lab Guide

HPC & Cluster Computing

Objective: Deploy a local testing environment using Infrastructure as Code (IaC). **Goal**: Install VirtualBox and Vagrant, and write a `Vagrantfile` that provisions 3 openEuler VMs (a Master Node and two Compute Nodes) with the correct network interfaces.

---

1. Introduction to the Tools

Before we can build an HPC cluster on the university's servers, we need a safe, local environment to experiment. We will use two tools:

- **VirtualBox**: A Type-2 Hypervisor that runs virtual machines on your personal laptop.
- **Vagrant**: A command-line utility for managing virtual machines. Instead of clicking through a GUI to create a VM, you write a configuration file (a `Vagrantfile`) and Vagrant automates the deployment. This is the foundation of *Infrastructure as Code*.

---

2. Installation

operating system (Windows, macOS, or Linux). Run the installer and follow the default prompts.

2.2 Download and Install Vagrant

Navigate to the Vagrant Downloads Page and download the installer for your operating system. Run the installer and follow the default prompts.

Once installed, open a Terminal (or PowerShell on Windows) and verify the installation:

```
vagrant --version
```

---

3. Creating the Vagrantfile

We are going to deploy an **openEuler** virtual machine that will act as our "Master Node".

Explanation: A `Vagrantfile` is written in Ruby syntax. It tells Vagrant which OS image (called a "box") to download, how much RAM to allocate, and how to configure the network interfaces.

1. Open your terminal/PowerShell and create a new directory for your HPC labs, then navigate into it:

```
mkdir hpc-labs
cd hpc-labs
```



```
Vagrant.configure("2") do |config|
  # We use a generic openEuler box
  config.vm.box = "generic/openeuler22"

  # Master Node (Control)
  config.vm.define "master" do |master|
    master.vm.hostname = "master"

    # NIC 1 is NAT (Internet) by default in Vagrant.
    # We define NIC 2 here as our internal cluster networ
    master.vm.network "private_network", ip: "10.0.0.1",

    master.vm.provider "virtualbox" do |vb|
      vb.memory = "2048" # 2 GB of RAM
      vb.cpus = 2
      vb.name = "hpc-master-node"
    end
  end

  # Compute Node 01
  config.vm.define "compute-01" do |node|
    node.vm.hostname = "compute-01"
    node.vm.network "private_network", ip: "10.0.0.101",
    node.vm.provider "virtualbox" do |vb|
      vb.memory = "1024" # 1 GB of RAM
      vb.cpus = 1
      vb.name = "hpc-compute-01"
    end
  end

  # Compute Node 02
  config.vm.define "compute-02" do |node|
```

```
    node.vm.hostname = "compute-02"
    node.vm.network "private_network", ip: "10.0.0.102",
    node.vm.provider "virtualbox" do |vb|
      vb.memory = "1024" # 1 GB of RAM
      vb.cpus = 1
      vb.name = "hpc-compute-02"
    end
  end
end
```

4. Deploying the Infrastructure

Now that our infrastructure is defined in code, we will deploy it.

Explanation: The `vagrant up` command reads the Vagrantfile, downloads the OS box from the internet (this might take a few minutes the first time), creates all 3 VMs in VirtualBox in sequence, sets up the networking, and boots them.

```
vagrant up
```

Once the command completes, you have a fully functional miniature cluster running in the background!

5. Accessing and Managing the Node

5.1 Logging In

Explanation: The `vagrant ssh` command automatically uses the private key Vagrant generated to log you securely into the virtual machine without needing a password.

```
vagrant ssh master
```

Verify your IP addresses inside the Master VM:

```
ip addr show
```

You should see `eth0` (NAT/Internet) and `eth1` (10.0.0.1 - Your cluster network).

5.2 Network Testing

Because Vagrant deployed the VMs onto the same Private Network, they can communicate with each other. From the Master node, verify you can reach the compute nodes:

```
ping -c 4 10.0.0.101
ping -c 4 10.0.0.102
```

Type `exit` to log out of the VM and return to your host terminal.

5.3 Managing the VM State

```
# To gracefully shut down the VM:
vagrant halt

# To turn it back on later:
vagrant up

# To completely delete the VM and its hard drive:
vagrant destroy
```

6. Submission

Take a screenshot showing your terminal window with:

1. The output of `vagrant status` showing all 3 nodes running.
2. An active `vagrant ssh master` session where you ran the `ping -c 4 10.0.0.101` command showing successful replies from the compute node.

Save this screenshot as `Week1_Lab_Surname.png`